AI-Powered Intelligent Query Resolution System

Week 1 Internship Progress Report: Research, Requirements Analysis, and Initial System Design

Team A

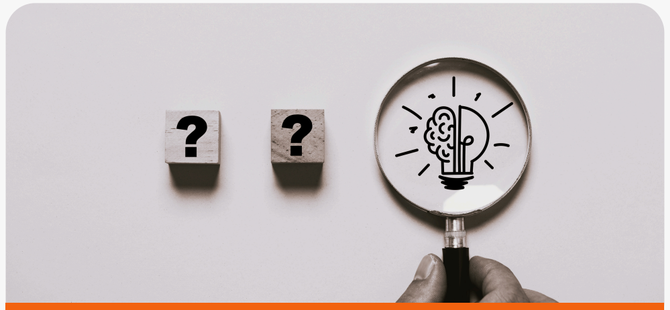
22/10/2026

Agenda

1. Introduction and Project Context
2. Research, Objectives, and Technology
3. System Design and Workflow
4. Progress and Next Steps

Introduction and Project Context

AI-Powered Intelligent Query Resolution System



Week 1 Progress Review

- AI-Powered Intelligent Query Resolution System — research and initial design.



Internship Details

- Intern/Team: Team A
- Mentor: Vivek Gautam
- Organization: Infosys.
- Reporting period: Week 1
- Submission date: 22 Oct 2026.

Project Summary

What we are building

- A domain-agnostic assistant that answers questions from uploaded documents.
- Designed to return grounded responses with supporting source references.

How users will interact

- Users can ask queries through text or browser-based **voice** input.
- Voice output is planned, with typed input as a fallback option.

Core technical approach

- **RAG** retrieves relevant passages before generating the final answer.
- A multi-agent layer separates understanding, retrieval, and response steps.

Week 1 emphasis

- This week established requirements, research, and initial system design only.



Problem and Need



Information is scattered

- Teams rely on manuals, policies, FAQs, and process guides.
- Finding the right detail often requires browsing multiple documents.

Search is often inefficient

- Keyword search can miss context or the exact phrasing used.
- Manual lookup increases time-to-answer and inconsistency in responses.

Conversational access adds value

- Natural-language Q&A helps users reach relevant passages faster.
- A domain-agnostic approach supports different document collections.

Research, Objectives, and Technology

Week 1 objectives

Requirements and scope alignment

- Confirm expected capabilities and define clear Week 1 boundaries.
- Document what is planned versus what is not yet implemented.

Core research and tooling review

- Build practical understanding of **RAG** for source-grounded answers.
- Review multi-agent patterns plus browser speech input/output options.

Design baseline for build phase

- Select initial candidates for ingestion, embeddings, vector search, and API.
- Draft architecture, orchestration flow, and data model for Week 2 work.



RAG Research Highlights

Why RAG was selected

- It grounds answers by pulling relevant source text first.
- This improves traceability versus relying only on model memory.

Indexing (offline) focus

- Prepare searchable chunks, embeddings, and metadata ahead of questions.
- This setup enables fast, semantic retrieval at query time.

Querying (online) focus

- Encode the question, retrieve top matches, then generate a response.
- Return answers with source references when retrieval is relevant.

Week 1 takeaway

- Research defined a practical baseline for Week 2 implementation work.



Technology Choices

Text preparation (chunking)

- Use recursive character splitting to keep paragraphs intact where possible.
- Add overlap so answers retain nearby context across chunk boundaries.

Semantic representation (embeddings)

- Generate vectors using sentence-transformers **all-MiniLM-L6-v2** as a compact baseline.
- Prioritize fast, local embeddings suitable for early retrieval experiments.

Storage and service layer

- Index chunks and metadata in **ChromaDB** for top-k dense retrieval.
- Expose upload and query endpoints through **FastAPI** for integration.

Status and validation

- These are initial design choices and will be confirmed during Week 2 build.

System Design and Workflow

4-Layer Architecture

Presentation layer (UI)

- Provides chat, upload controls, and optional browser voice I/O.
- Surfaces source transparency so users can trace answers back.

API layer (services)

- Exposes endpoints for document upload, queries, and sessions.
- Returns answers plus citations for the selected knowledge base.

Orchestration layer (logic)

- Coordinates intent detection, retrieval, clarification, and generation.
- Uses conversation context to handle follow-up questions reliably.

Knowledge and data layer

- Parses files, chunks text, generates embeddings, and indexes vectors.
- Stores metadata and logs to support traceability and debugging.

End-to-End Workflow

Ingestion pipeline (offline)

- Validate uploads, extract text, clean content, and create chunks.
- Generate embeddings and index chunks with source metadata.

Question submission (online)

- Users ask via typed chat or supported browser voice input.

Retrieve, then answer

- Retrieve top relevant chunks and draft a grounded response.
- Attach citations and treat low-relevance matches cautiously.

Agent Responsibilities

Conversation context

Uses relevant prior turns to rewrite follow-up questions clearly.

Intent and routing

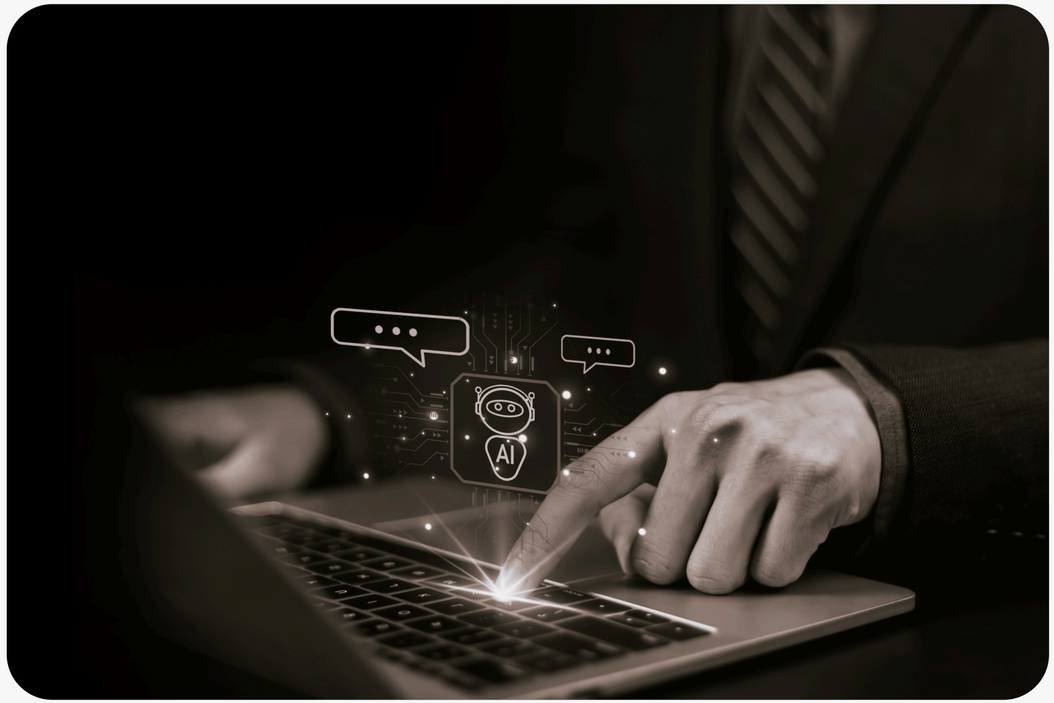
Classifies the question type and selects the next processing path.

Evidence and clarification

Retrieves ranked chunks; asks a focused follow-up if ambiguous.

Grounded response output

Generates the final answer with supporting citations to sources.



Data & Ingestion Plan

Core data entities

- Document and Chunk capture text segments with file provenance.
- Embedding links each chunk to a searchable semantic vector.
- Session and Message preserve conversational context when needed.
- Retrieval Log stores queries, chunk IDs, and relevance scores.

Planned ingestion pipeline

- Initial file support: PDF, DOCX, TXT, and FAQ-style CSV.
- Stages: validate, parse, clean, chunk, then embed and index.
- Retain **source metadata** to enable citations and debugging.
- Traceability supports later evaluation and retrieval tuning.

Progress and Next Steps

Week 1 Progress Summary

Aligned on scope and boundaries

Week 1 documented research/design; no build claims

Researched RAG and agent patterns

Defined indexing vs querying phases and routing ideas

Drafted initial system design

Layered architecture, workflow, and agent responsibilities

Selected candidate tech stack

FastAPI, ChromaDB, MiniLM embeddings, Web Speech API

Outlined ingestion and data model

Validate-parse-clean-chunk-embed-index; metadata retained

Captured risks and next validation needs

PDF parsing, chunk trade-offs, browser voice variability

Week 2 Plan



Environment and workspace

- Set up Python, dependencies, and a clean VS Code project structure.
- Confirm package versions and compatibility before building modules.



Build the ingestion baseline

- Implement loaders, then add cleaning and chunking with metadata retained.
- Generate embeddings and create a persistent ChromaDB vector index.



Validate with small tests

- Run a small set of queries and review retrieved chunks for relevance.
- Log real errors, fixes, and outcomes; avoid unmeasured performance claims.